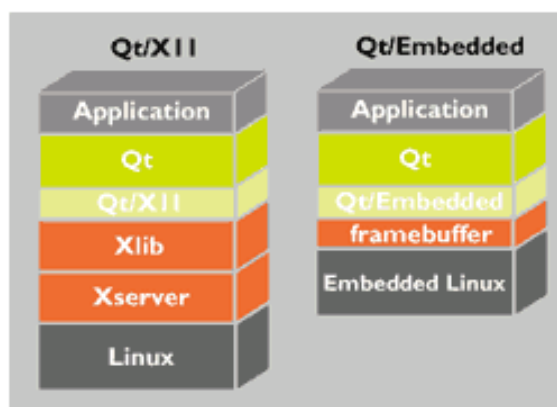


ภาคผนวก ข.

ชุดไลบรารี Qt/Embedded

ข.1 ภาพรวมของQt/Embedded

Qt/Embedded เป็นชุดเครื่องมือภาษา C++ เพื่อใช้ในการสร้างส่วนติดต่อกับผู้ใช้ และพัฒนาแอปพลิเคชันสำหรับอุปกรณ์แบบฝังตัว โดยที่สามารถทำงานกับหน่วยประมวลผลได้หลายแบบ ที่สามารถทำงานกับระบบปฏิบัติการลินุกซ์แบบฝังตัว (Embedded Linux) แอปพลิเคชันของ Qt/Embedded จะเขียนโดยตรงกับส่วนของเฟรมบัพเฟอร์ (Framebuffer) ทำให้ไม่มีความต้องการในส่วน of ระบบ X-Window และชุดของไลบรารี, Qt/Embedded ได้รวมเครื่องมือต่างๆ เพื่อเพิ่มความเร็ว และความสะดวกในการพัฒนาแอปพลิเคชัน โดยใช้สภาพแวดล้อมในการเขียนโปรแกรมแบบเดียวกับที่ใช้ในการพัฒนาแอปพลิเคชันบนระบบปฏิบัติการวินโดวส์ และ ยูนิกซ์ ซึ่งใช้โอพีไอของ Qt ตัวเดียวกัน

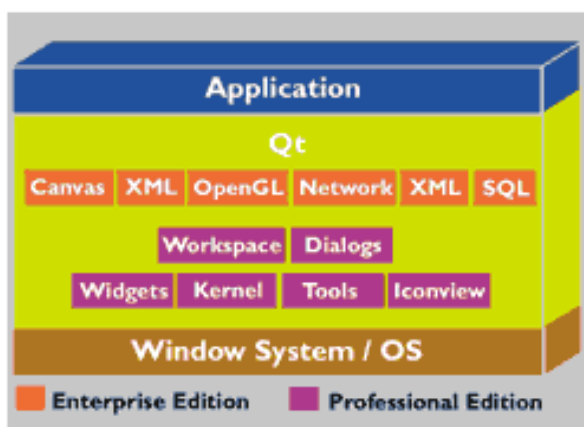


รูปที่ ข-1 แสดงการติดต่อเฟรมบัพเฟอร์โดยตรงของ Qt/Embedded

ทำให้จำเป็นต้องใช้ระบบ Xlib/Xserver

Qt/Embedded นั้นมีการเตรียมโอพีไอ และชุดเครื่องมือต่างๆ เหมือนกับที่ใช้ใน Qt/X11 (ชุดเครื่องมือของ Qt ที่ใช้พัฒนามบนระบบปฏิบัติการลินุกซ์ และ ยูนิกซ์), Qt/Windows (ชุดเครื่องมือของ Qt ที่ใช้พัฒนามบนระบบปฏิบัติการวินโดวส์) และ Qt/Mac (ชุดเครื่องมือของ Qt ที่ใช้พัฒนามบนระบบปฏิบัติการแมคโอเอส) Qt/Embedded ยังรวมคลาส รวมถึงเครื่องมือต่างๆ ที่สนับสนุนการพัฒนาสำหรับอุปกรณ์แบบฝังตัวด้วยเช่นกัน ซึ่งคลาสต่างๆ ของ Qt นั้นสามารถช่วยให้ผู้ใช้ที่ต้องการพัฒนาแอปพลิเคชันลดภาระงานของผู้ใช้ และจัดเตรียมส่วนเชื่อมต่อต่างๆ เพื่อเพิ่มความรวดเร็วในการเรียนรู้

ข.2 ข้อเด่นของการใช้ Qt/Embedded ในการพัฒนา



รูปที่ ข-2แสดงโมดูลต่างๆที่ Qt มีซึ่ง Qt/Embedded
มีโมดูลเหล่านี้ทั้งหมด รวมไปถึงระบบแสดงผลหน้าจอแบบวินโดวส์

Qt/Embedded มีการนำข้อดีต่างๆมาให้กับผู้พัฒนาในการสร้างแอปพลิเคชันที่สามารถใช้ได้กับหลายแพลตฟอร์ม โดยที่เฟรมเวิร์คในการพัฒนาระบบแบบฝังตัวนี้มีสิ่งที่น่าสนใจคือ

- มีเอฟไอโอเดียวกันกับที่ใช้บนคอมพิวเตอร์เดสทอปทั่วไป โดยที่ผู้พัฒนาสามารถเรียนรู้เอเอฟไอโอเพียงแค่ตัวเดียว แต่สามารถทำงานในสภาพแวดล้อมต่างๆได้ ไม่ว่าจะเป็นบนคอมพิวเตอร์เดสทอปทั่วไป (ที่ใช้ระบบปฏิบัติการวินโดวส์, ลินุกซ์, แมค โอเอส) หรือบนสภาพแวดล้อมของระบบปฏิบัติการลินุกซ์แบบฝังตัว
- ขยายความสามารถของ Qt ไปยังระบบแบบฝังตัว ซึ่งทำให้แอปพลิเคชันที่ได้ออกมามีความต้องการหน่วยความจำที่น้อย
- บรรจุระบบหน้าต่าง ต่างๆของตัวเอง โดยที่ไม่ต้องการระบบย่อยที่ใช้ในด้านกราฟิกแสดงผล Qt/Embedded ทำการใส่ส่วนของกราฟิกลงในเฟรมบัพเฟอร์ไว้เรียบร้อยแล้ว
- เป็นสัดส่วนอย่างชัดเจน ทำให้สามารถเก็บหน่วยความจำไว้ใช้อย่างคุ้มค่า โดยการปิดการทำงานของส่วนที่ไม่ใช้ลงไป
- เพิ่มความสามารถโดยการรวมกับ Qtopia ซึ่งเป็น ส่วนติดต่อกับผู้ใช้ในด้านต่างๆ ไม่ว่าจะเป็นส่วนจัดการข้อมูลส่วนตัว (PIM – Personal Information Management), เว็บ, ความบันเทิง การทำงานต่างๆ และยังสนับสนุนความสามารถของ จาวา รวมถึงสามารถส่งผ่านข้อมูลข้ามระบบปฏิบัติการที่อยู่บนคอมพิวเตอร์เดสทอปได้
- เพิ่มความสามารถยิ่งขึ้นโดยการรวมกับจาวา นั่นคือ QtAWT ซึ่งทำให้แอปพลิเคชันที่ใช้ Qt/Embedded ใช้ความสามารถของจาวาได้ โดยที่เมื่อจาวาส่งให้ AWT เรียก

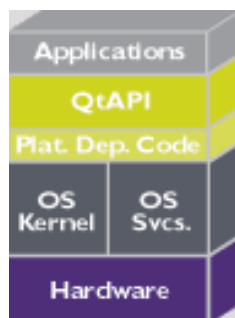
แอปพลิเคชันที่ทำงานบนระบบปฏิบัติการลินุกซ์สำหรับระบบฝังตัว, QtAWT จะส่งการเรียกนั้นไปยัง Qt แทนที่จะส่งไปที่ Xlib

- มีซอร์สโค้ดแบบสมบูรณ์ ทำให้ผู้พัฒนา สามารถปรับเปลี่ยนความสามารถให้เพิ่มมากยิ่งขึ้นได้

ข.2 Qt

Qt เป็นแอปพลิเคชันเฟรมเวิร์คที่ใช้ในการพัฒนา โดยใช้ภาษา C++ ในการพัฒนา ซึ่งเป็นของบริษัทประเทศนอร์เวย์ที่มีชื่อว่า Trolltech โดยที่จุดเด่นที่สำคัญคือ ชุดโปรแกรมที่เขียนโดยใช้เฟรมเวิร์ค Qt สามารถนำไปทำงานได้ในหลายแพลตฟอร์ม โดยการนำชุดคำสั่งนั้น มาคอมไพล์ใหม่ ให้เหมาะสมกับแต่ละแพลตฟอร์ม ซึ่งสามารถทำงานบนแพลตฟอร์มต่างๆดังนี้

- Qt/Windows คือทำงานบนระบบปฏิบัติการวินโดวส์ 95/98/Me, NT4, 2000 และ XP
- Qt/X11 คือทำงานบนระบบปฏิบัติการลินุกซ์, ซอลาริส (Solaris), เฮชพี-ยูเอ็กซ์ (HP-UX), ไอริคส์ (IRIX), เอไอเอ็กซ์ (AIX), และระบบปฏิบัติการอื่นๆที่เกี่ยวข้องกับยูนิกซ์
- Qt/Mac คือทำงานบนระบบปฏิบัติการแอปเปิลแมคโอเอส เท็น (Apple Mac OS X)
- Qt/Embedded คือทำงานบนระบบปฏิบัติการลินุกซ์สำหรับระบบฝังตัว



รูปที่ ข-3 แสดงส่วนชั้นของการทำงานของ Qt

จุดแข็งของ Qt ที่มีนั้นทำให้ผู้ที่พัฒนาเขียนแอปพลิเคชันเพียงชุดเดียว แต่สามารถทำงานได้ในแพลตฟอร์มต่างๆได้โดยการคอมไพล์ให้เหมาะสมเท่านั้น รวมถึง Qt เป็นชุดพัฒนาที่ออกมาเป็นแบบเชิงวัตถุ (Object-Oriented) แบบสมบูรณ์ ซึ่งข้อดีของ Qt ก็คือ

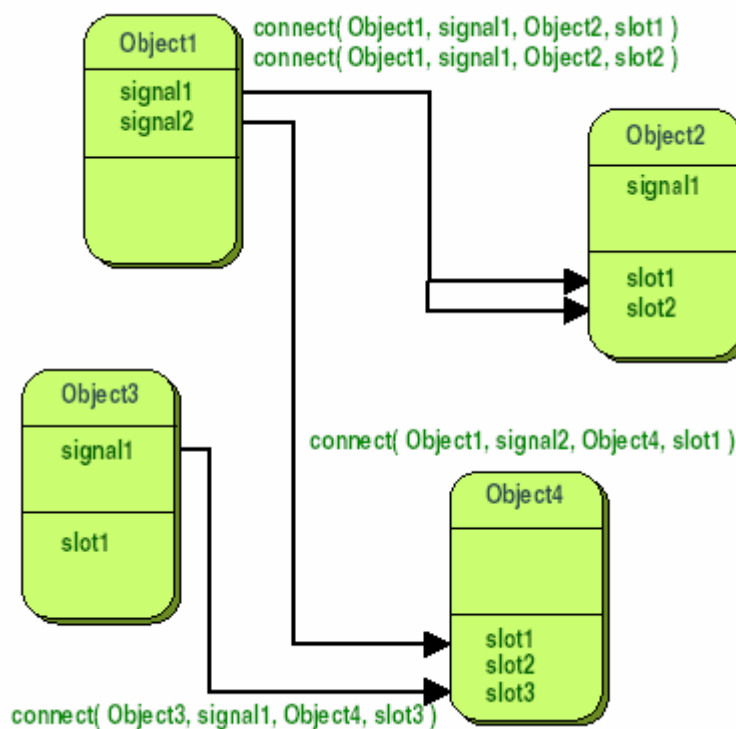
- สามารถเข้ากับแพลตฟอร์มต่างๆได้ ทำให้ประหยัดเวลาในการพัฒนา เพื่อใช้กับแพลตฟอร์มต่างๆ
- สามารถรวมกับ ทูลคิทของ Motif ได้
- มีการจัดการเอฟไอโอที่เฉพาะเจาะจงของทั้งยูนิกซ์, วินโดวส์ และแมค รวมถึงเอฟไอโอสำหรับการจัดการไฟล์, เน็ตเวิร์ค, การจัดการโปรเซส, เทรคคิง, การเข้าถึงระบบฐานข้อมูล และอื่นๆ
- ทำให้ลดค่าใช้จ่ายในการพัฒนาและแก้ไข เนื่องจากสามารถใช้ได้กับทุกแพลตฟอร์ม

- ไม่มีความต้องการเวอร์ชั้วแมชชีน (Virtual Machines) หรือชั้นของการทำอิมูเลชัน (Emulation) เพราะ Qt ติดต่อดโดยตรงกับส่วนกราฟฟิกระดับล่าง ทำให้มีความเร็วในการทำงาน

ข.3 Signals and Slots

Signals and Slots เป็นส่วนที่ใช้ในการติดต่อกันระหว่างอ็อบเจ็กต์ต่างๆภายในแอปพลิเคชัน โดยนำมาทดแทนการใช้ฟังก์ชัน callback และ message map ที่ใช้ในทูลคิทแบบเก่า ซึ่งไม่มีความสะดวก, เกิดความผิดพลาดได้ง่าย, ไม่มีความยืดหยุ่น และไม่เป็นการออกแบบเชิงวัตถุ (Object-Oriented) ซึ่ง Signals and Slots นั้นมีความปลอดภัย, รวดเร็ว, ยืดหยุ่น และเป็นการออกแบบเชิงวัตถุ

ในการเชื่อมต่อชุดคำสั่งกับปุ่มใดๆนั้น ด้วยการใช้การทำงานของ callback นั้น จะต้องมีการส่งผ่านพอยต์เตอร์ไปยังฟังก์ชัน ไปยัง ปุ่ม เมื่อปุ่มถูกกด ฟังก์ชันก็จะถูกเรียกขึ้นมา ในทูลคิทแบบเก่านั้น ไม่มีการรับประกันได้ว่า ชนิดของตัวแปรที่ถูกต้องจะส่งไปยังฟังก์ชันเป้าหมายเมื่อมันถูกเรียก ซึ่งมักจะทำให้เกิดการพลัดขึ้นได้ ปัญหาอีกอย่างหนึ่งที่เกิดขึ้นกับวิธี callback ก็คือมันซ่อนฟังก์ชันการทำงานต่างๆไว้ภายใต้ส่วนของส่วนติดต่อกับผู้ใช้ได้ไม่ก่อนสะดวก ทำให้ยากที่จะพัฒนาคلاسต่างๆให้แยกต่อกัน



รูปที่ ข-5 โครงสร้างโดยทั่วไปในการเรียกของวิธี Signals and Slots

Signals and Slots นั้นทำงานแตกต่างออกไป ส่วนวิดเจ็ต (Widget) ของ Qt นั้นจะส่ง Signal เมื่อมีเหตุการณ์เกิดขึ้น ยกตัวอย่างเช่น ปุ่มกดได้ส่ง Signal “clicked” เมื่อมันถูกกด โปรแกรมเมอร์สามารถเลือกการติดต่อกับ Signal ด้วยการเลือกฟังก์ชัน (เรียกว่า Slot) และเรียกคำสั่ง connect() เพื่อเชื่อมการทำงานระหว่าง Signal และ Slot วิธีนี้ทำให้ไม่จำเป็นต้องให้คลาสต่างๆรู้จักกับคลาสอื่น ซึ่งทำให้ง่ายต่อการพัฒนา วิธี Signals and Slots นั้นมีความปลอดภัย ความผิดพลาด (Error) จะถูกส่งออกมาเพียงแค่การเตือน ไม่ใช้การทำงานผิดพลาดทั้งหมด

ตัวอย่างที่เห็นได้ชัด เช่น มีปุ่มออกจากโปรแกรม ซึ่งมี Signal ‘clicked()’ และเชื่อมต่อกับ Slot ‘quit()’ ของแอปพลิเคชันหลัก เมื่อผู้ใช้งานกดปุ่มออก จะทำให้แอปพลิเคชันนั้นปิดตัวลงไป ในการเขียนโค้ด เขียนได้โดยใช้คำสั่ง

```
connect( button, SIGNAL(clicked()), qApp, SLOT(quit()) );
```

การเชื่อมต่อนั้น สามารถเพิ่มเข้าไปหรือเอาออกได้ตลอดเวลาในขณะที่รันแอปพลิเคชัน วิธี Signals and Slots นั้นทำให้ได้ความสามารถเชิงวัตถุของภาษา C++ ได้อย่างเต็มที่ ซึ่งวิธีนี้ สามารถใช้ได้ในทุกส่วนของคลาส ไม่ว่าจะเป็นในส่วนของ Public, Protected หรือแม้กระทั่ง Private